

Infraestructuras y Procesos de Soporte

Examen - Docker y Kubernetes

Instrucciones generales

- **Antes de comenzar el examen tendrás que ejecutar un script que se encuentra en la ruta que el profesor pondrá en la pizarra. Sigue las instrucciones que se den porque de lo contrario, no podrás realizar el examen adecuadamente.**
 - Descarga el paquete del examen desde Campus Virtual y descomprímelo (donde tengas acceso).
 - Debes trabajar desde el directorio `alumnos\`.
 - Al terminar, entrega todos el contenido de `alumnos\` comprimido en un ZIP con tu nombre.
 - Únicamente puedes consultar el material disponible en Campus Virtual. No está permitido usar Internet.
 - El código Flask en `app\` ya está escrito. No lo modifiques.
 - Utiliza exactamente los nombres que se indican en el enunciado para los diferentes elementos (ficheros, despliegues, servicios, etc.)
 - Todo fichero cuya estructura no esté bien formada, no será tenido en cuenta en la corrección.
-

Contexto

Vamos a contenerizar y desplegar con Kubernetes una aplicación web en Python (Flask) que sirve una página de bienvenida configurable.

La aplicación (`app/app.py`) hace dos cosas:

1. Lee `/config/settings.ini` para obtener título y autor.
 2. Lee la variable de entorno `APP_ENV` para mostrar el entorno activo.
-

PARTE 1 - Docker · 4 puntos ·

Ejercicio 1.1 - Dockerfile (2 puntos)

Crea un fichero llamado `Dockerfile` con los siguientes requisitos:

1. Imagen base: `webapp-base:latest`
2. Directorio de trabajo: `/app`
3. Copiar `requirements.txt` e instalar dependencias con `pip`
4. Copiar el resto del código
5. Exponer el puerto `5000`
6. Arrancar con: `flask run --host=0.0.0.0 --port=5000`

Construye a partir del `Dockerfile` una imagen con la etiqueta `webapp:latest`

Verificación:

Crea un contenedor a partir de la imagen con mapeo de puertos 5000:5000.

Abre <http://localhost:5000> - debe aparecer la página HTML. Para detener: **Ctrl+C**.

Ejercicio 1.2 - Docker Compose (2 puntos)

Crea un fichero llamado `docker-compose.yml` con dos servicios:

Servicio	Descripción
<code>web</code>	App Flask, con la imagen anteriormente construida
<code>proxy</code>	<code>nginx:alpine</code> , proxy inverso al servicio <code>web</code>

Requisitos:

- `web`: variable de entorno `APP_ENV=development`, volumen `./logs:/app/logs`
- `proxy`: puerto `80:80`, monta `nginx/nginx.conf` en `/etc/nginx/conf.d/default.conf` (solo lectura), dependen de `web`
- Ambos en red `appnet` con driver `bridge`

Verificación:

Lanza docker compose.

Abre <http://localhost> - debe aparecer la página a través del proxy.

`docker compose down`

PARTE 2 - Kubernetes · 6 puntos ·

Ejercicio 2.1 - Deployment y Service (2 puntos)

Crea un fichero `deployment.yml` con un Deployment y un Service.

Deployment `webapp-deployment`:

- 2 réplicas
- Imagen `webapp:latest` con `imagePullPolicy: Never`
- Variable de entorno `APP_ENV=production`
- Etiqueta `app: webapp`

Service `webapp-service`:

- Tipo `NodePort`, puerto del contenedor `5000`, puerto del nodo `30080` para etiqueta `app: webapp`

Verificación:

Aplica `deployment.yml`.

Espera a que los dos pods estén en `Running`.

Abre `http://localhost:30080`.

Nota:

Si estás usando minikube es necesario antes cargar la imagen usada en el deployment con `minikube image load webapp:latest` y cuando vayas a comprobar los servicios tendrás que usar `minikube service webapp-service`.

Ejercicio 2.2 - ConfigMap (2 puntos)

2.2.a Crea un fichero `configmap.yml` con un ConfigMap llamado `app-config` que contenga una clave llamada `settings.ini` con el siguiente contenido:

```
[app]
title = Mi App de Examen
author = PON AQUI TU NOMBRE Y APELLIDOS
```

2.2.b En `deployment.yml` añade el volume `config-volume` y su volumeMount montado en el directorio `/config` del contenedor.

Verificación:

Aplica `configmap.yml`.

Aplica `deployment.yml`.

Abre `http://localhost:30080` - debe mostrar **"Mi App de Examen"** y tu nombre.

Ejercicio 2.3 - Volumen persistente (2 puntos)

2.3.a Crea un fichero `pvc.yml` con un PVC llamado `app-logs-pvc`, modo `ReadWriteOnce`, `100Mi`.

2.3.b En `deployment.yml` añade el volume `logs-volume` y su volumeMount en `/app/logs`.

Verificación:

Aplica `pvc.yml`.

Aplica `deployment.yml`.

Comprueba que el PVC aparece con estado `Bound`.